

C++ 分布式百万并发弹幕系统完整设计方案

C++ 分布式百万并发弹幕系统完整设计方案

一、核心指标定义

目标：100 万同时在线用户（DAU 更高）、单直播间峰值弹幕 QPS ≥ 5 万、端到端延迟 $< 100\text{ms}$ 、消息不丢不重、水平无限扩容 技术栈主体：C++17/20 + epoll/IO_URING + Redis Cluster + Kafka + gRPC + Consul/Nacos + 分片一致性哈希 + WebSocket/QUIC

关键容量拆解

1. 百万在线：按直播间分片打散，单节点承载 2~5w 在线长连接；
2. 弹幕流转链路：上行发送 $\rightarrow$ 消息队列削峰 $\rightarrow$ 分发服务推送至所有在线观众；
3. 约束：禁止单进程持有全量直播间连接，无单点瓶颈。

二、整体架构分层（五层架构）

plaintext

代码块

- ```
1 客户端(Web/App)
2 ↓ QUIC/WebSocket 长连接接入层
3 接入网关层 Gateway(C++)
4 ↓ gRPC 内部RPC
5 消息分发层 Dispatcher(C++)
6 ↓ Kafka 异步削峰
7 弹幕业务处理层 Worker(C++)
8 ↓ Redis Cluster (房间在线用户、弹幕持久化、限流、去重)
```

整体架构横向无状态，所有服务可无限加机器扩容。

### 三、各层详细设计（C++ 针对性优化）

#### 1. 接入网关 Gateway（长连接层，C++ 核心）

职责

- 维护百万用户 WebSocket/QUIC 长连接；

- 心跳保活、断线重连、协议解析、鉴权、IP 限流；
- 上行弹幕数据包初步校验，转发给分发层；
- 接收下游推送的弹幕，批量下发给本节点在线观众。

## C++ 技术选型

### 1. 网络框架

- 轻量自研 reactor: epoll (Linux) / IO\_URING, 多 Reactor 多线程模型；
- 备选成熟开源: Boost.Asio、muduo、brpc (百度 C++ RPC 框架, 自带连接池、负载均衡)；

1. 推荐 brpc: 自带批量发送、连接复用、流量控制, 适配内网 RPC。

2. 连接分片 单个 Gateway 进程拆多 Worker 线程, 每个线程独立 Reactor, 互不锁竞争; 单台物理机部署多 Gateway 实例, 单实例承载上限 3~5w 长连接 (受文件句柄 fd 限制)。

## 关键优化点 (C++ 专属)

1. 内存池: 预先分配收发缓冲区, 避免频繁 `new/malloc` 造成堆碎片;
2. 批量发包 (writev): 同一个直播间多条弹幕合并一次系统调用下发, 大幅降低 syscall 耗时;
3. 无锁队列: 线程间消息传递使用 spsc 无锁环形队列, 摒弃 `pthread_mutex`;
4. fd 上限调优: Linux `ulimit -n` 调至 100 万+, 内核 `tcp_tw_reuse` 打开;
5. 协议轻量化: 二进制 Protobuf 代替 JSON 序列化, 单包体积减少 60%+。

## 路由规则

每个用户长连接绑定 `Gateway实例ID + ConnID`, 注册到 Consul 服务发现。

## 2. 消息分发层 Dispatcher (房间路由中枢)

### 核心能力

1. 接收 Gateway 上行弹幕, 根据房间 ID 做一致性哈希分片;
2. 同一房间弹幕路由到固定一组 Worker 处理, 避免跨节点乱序;
3. 广播路由计算: 查询该房间所有在线用户分布在哪些 Gateway 节点, 批量推送弹幕。

### 分片策略 (水平扩容核心)

对 `room_id` 做哈希取模, 把直播间打散到 N 组 Worker 集群:

- 新增 Worker 节点仅少量房间迁移, 平滑扩容, 无抖动;
- 每个分片独立消费 Kafka 一个 partition, 天然并行。

## 限流控制 (防刷屏)

单用户每秒弹幕条数阈值、单房间总 QPS 阈值，超限直接丢弃，Redis 做计数器原子递增（INCR+EXPIRE）。

### 3. Kafka 削峰层（异步解耦必选）

#### 作用

1. 直播间瞬间爆发弹幕（主播高光时刻），QPS 突增 10 倍，Kafka 缓冲流量，避免 Worker 被打挂；
2. 异步解耦：Gateway 同步上行不阻塞，下游 Worker 慢慢消费；
3. 消息持久化：可回溯历史弹幕，用于回放、数据分析。

#### 部署配置

- Topic 按直播间分片分区，partition 数量  $\geq$  Worker 消费实例数；
- 每条消息 Key=room\_id，同房间消息进入同一 partition，保证弹幕时序不乱；
- C++ 客户端：librdkafka（高性能官方 C 库，C++ 封装简单）。

### 4. Worker 业务处理层（C++）

#### 业务逻辑

1. 消费 Kafka 弹幕消息；
2. 敏感词过滤（AC 自动机多模式匹配，C++ 实现，百万词库毫秒检索）；
3. 弹幕频率校验、等级权限校验、屏蔽用户过滤；
4. 组装最终推送数据包，下发给对应所有 Gateway 网关；
5. 热点弹幕计数、点赞统计、高频弹幕合并（同一条弹幕重复发送合并推送）。

#### C++ 性能优化

1. AC 自动机静态预加载敏感词，常驻内存，无运行时加载开销；
2. 字符串处理用 `std::string_view`，零拷贝解析数据包；
3. 批量 RPC 推送：一次 gRPC 批量发给一个 Gateway 上该房间所有在线连接，减少 RPC 调用次数。

### 5. Redis Cluster 存储层（3 大核心用途）

#### 1) 在线用户路由表

KEY=room:{room\_id} Hash 结构：gateway\_instance\_id -> [conn\_id列表] 记录每个房间当前所有观众分布在哪些网关，推送时直接批量寻址。

#### 2) 限流计数器

plaintext

代码块

```
1 KEY=user:rate:{uid} # 用户每秒发弹次数
2 KEY=room:qps:{room_id} # 房间总弹幕QPS
```

Redis 原子 INCR + 过期时间，分布式限流，多实例限流统一。

### 3) 弹幕缓存 + 历史回放

- 每个房间保留最近 N 条弹幕（List 固定长度 LPOP 裁剪），新进入直播间用户拉取历史滚动弹幕；
- 高频热门房间可开启本地 LRU 内存缓存，减少 Redis 访问。

### Redis 部署

3 主 3 从 Cluster，开启 Pipeline 批量命令，C++ 客户端 hiredis 异步连接池。

## 四、完整消息流转两条链路

### 上行链路（观众发弹幕）

观众 APP/Web → QUIC/WebSocket → Gateway → Protobuf 解析校验 → gRPC 推送 Dispatcher → 按 room\_id 哈希分片 → 投递 Kafka 对应 partition

### 下行链路（全员推送）

Worker 消费 Kafka → 敏感词过滤 + 校验 → Redis 查房间所有网关列表 → 批量 gRPC 下发各 Gateway → Gateway 批量 writev 发包 → 多个终端同时收到弹幕

## 五、百万在线扩容计算（容量规划）

### 1) Gateway 长连接节点

单实例稳定承载 4w 在线长连接 100w 在线总实例数 =  $100/4 = 25$  个 Gateway 实例 多机部署，单机部署 2~3 实例，横向加机器无限扩容。

### 2) Worker 处理节点

单 Worker 消费 partition 处理 QPS 1w 峰值总 QPS 5w → 部署 6~8 个 Worker，预留冗余。

### 3) Kafka 集群

partition 数量与 Worker 一一对应，broker 3 节点起步，磁盘机械盘即可。

### 4) Redis Cluster

6 节点（3 主 3 从），热点房间拆分 key，避免大 key 阻塞。

## 六、高可用设计（无单点故障）

1. 服务发现 Consul/Nacos Gateway/Dispatcher/Worker 全部注册服务，节点宕机自动剔除，新节点自动加入；
2. 网关无状态 用户断线重连可接入任意 Gateway，重新注册 room 映射，会话无绑定；
3. Kafka 重试机制 消费失败本地重试 3 次，仍失败写入死信队列，人工排查不阻塞正常弹幕；
4. Redis 主从切换 主节点宕机自动升从为主，不中断路由查询；
5. 降级策略
  - 超高并发：关闭敏感词精细过滤，只做关键词粗筛；
  - 房间过载：新观众禁止发弹幕，仅只读；
  - Redis 抖动：本地内存缓存临时兜底路由表。

## 七、C++ 关键代码模块伪代码示例

### 1. 网关批量下发弹幕（writev 零拷贝批量发包）

cpp

运行

代码块

```
1 // 同一个直播间多个连接，聚合多个缓冲区一次系统调用下发
2 struct ConnBuffer {int fd;
3 iovec vec[2];};
4 std::vector<ConnBuffer> batch_list; // 填充多个用户待发二进制弹幕包
5 for(auto& conn : room_conns) {auto pkt = build_danmaku_pkt(danmaku_data);
6 conn.vec[0].iov_base = pkt.data();
7 conn.vec[0].iov_len = pkt.size();
8 batch_list.push_back(conn);} // 批量 writev，一次 syscall 发给多个 fd
9 for(auto& item : batch_list) {writev(item.fd, item.vec, 1);}
```

### 2. AC 自动机敏感词过滤（C++ 高性能）

cpp

运行

代码块

```
1 class ACAutomaton {public: void insert(const std::string& word); void build();
2 std::vector<std::string> match_all(const std::string_view&
3 text); private: struct Node { int next[128], fail; bool end; };
```

```
3 std::vector<Node> nodes;}; // 全局单例预加载所有敏感词，启动时初始化static
 ACAutomaton g_ac;
```

### 3. librdkafka 异步生产消息

cpp

运行

### 代码块

```

1 rd_kafka_t* producer = rd_kafka_new(RD_KAFKA_PRODUCER, conf, &errstr, 1024);
2 rd_kafka_topic_t* rkt = rd_kafka_topic_new(producer, "danmaku_topic", NULL);//
 key=room_id保证同房间进入同一partitionrd_kafka_produce(rkt,
 RD_KAFKA_PARTITION_UA,
3 RD_KAFKA_MSG_F_COPY,
4 buf.data(), buf.size(),
5 room_key.c_str(), room_key.size(), nullptr);rd_kafka_flush(producer, 10);

```

## 八、性能瓶颈规避 (C++ 特有坑)

1. 锁竞争 长连接线程独立 Reactor，每个线程私有连接集合，全局只用无锁队列通信，杜绝全局大锁；
2. 内存泄漏 & 碎片 收发缓冲区内存池复用，长连接断开时回收缓冲区，不用频繁 new；
3. TCP 小包风暴 开启 TCP\_NODELAY=false，小包合并；Gateway 层批量聚合多条弹幕一次下发；
4. CPU 亲和性 C++ 进程绑定 CPU 核心（sched\_setaffinity），减少线程上下文切换。

## 九、部署架构拓扑简图

plaintext

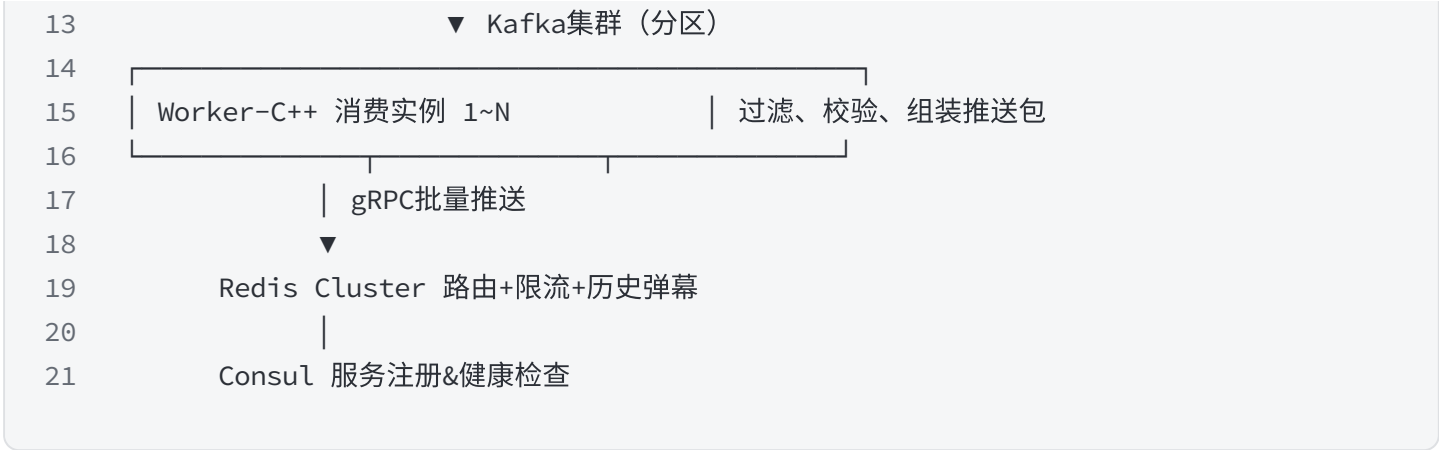
代码块

代码块

```
graph TD
 C[客户端集群] --> L[多机负载均衡 LVS/nginx]
 L --> G1[Gateway-C++-01]
 L --> G2[Gateway-C++-02]
 L -.-> G25[... 25个Gateway实例]
 G1 --> D[Dispatcher 分发层 C++]
 G2 --> D
 G25 --> D
 G1 --> P[长连接维护+协议解析]
 G2 --> P
 G25 --> P
 D --> R[房间哈希分片、路由计算]
```

The diagram illustrates the network architecture flow:

- 客户端集群** (Client Cluster) connects to **多机负载均衡 (LVS/nginx)** (Multi-machine Load Balancing).
- The load balancer distributes traffic to **Gateway-C++-01**, **Gateway-C++-02**, and **... 25个Gateway实例** (25 Gateway instances).
- Each Gateway instance handles **长连接维护+协议解析** (Long connection maintenance + protocol parsing).
- Traffic from the Gateways is sent to the **Dispatcher 分发层 (C++)** (Dispatcher distribution layer).
- The Dispatcher layer then routes traffic to **房间哈希分片、路由计算** (Room hash sharding, routing calculation).



## 十、扩展迭代方案

- 1. 超大规模（千万在线）：新增区域网关层，按地域就近接入，降低跨城延迟；
- 2. 弹幕分区冷热分离：冷门房间 Redis 路由下沉本地内存，减少 Redis 压力；
- 3. 支持弹幕上屏优先级：礼物弹幕、房管弹幕插队置顶，C++ 内存排序实现；
- 4. 接入监控：Prometheus + Grafana 采集各层 QPS、延迟、连接数、丢包率，C++ 埋点 brpc 内置 metrics。

需要我输出某一层完整可编译的 C++ 精简 Demo（Gateway WebSocket 服务 + Kafka 生产示例）吗？

快速

超能模式Beta

PPT 生成

AI 表格

图像生成

帮我写作

更多